

TP2: Críticas Cinematográficas - Grupo 02

Introducción

El problema a resolver en este trabajo era: predecir si una crítica cinematográfica en español es positiva o negativa.

Inicialmente, el dataset contaba con 50.000 registros y 3 columnas. Las columnas son: id del registro, la reseña como texto y su sentimiento (positivo o negativo)

El preprocesamiento del dataset varía según el modelo utilizado por lo que entraremos en detalles al explicar cada modelo. Sin embargo, lo que todos tienen en común es el filtrado de reseñas en otros idiomas. Utilizamos la librería `langid` para mantener solamente aquellas reseñas que están escritas en español, quedando finalmente 48.179 registros, que representa un 3,6% del dataset original.

También observamos que el dataset estaba balanceado equitativamente antes del filtrado por idioma. Luego del filtrado, ya no era una igual, pero no provocó un desbalance considerable.

Descripción de Modelos

Bayes Naive

Primero intentamos utilizar el modelo de Naive Bayes de manera simple, utilizando como vectorizador TF-IDF, sabiendo que seguramente traería mejores resultados que un vectorizador más simple como el de conteo.

Para el pre procesamiento, normalizamos el texto de las reseñas, pasando todo a minúscula y eliminando stop words.

Además, utilizamos K-Fold CV con $K = 5$ para optimizar hiperparametros en el modelo. Con esto obtuvimos un buen baseline. A partir de ahí tratamos de mejorarlo y fuimos agregando cosas.

Primero cambiamos el alcance de las stopwords, nos dimos cuenta que podríamos estar quitando palabras clave que cambiaban el significado de una oración, por lo que hicimos ese ajuste. Además, agregamos al pipeline de optimización de hiperparametros el vectorizador, para obtener aún mejores resultados.

Por último, agregamos lematización al pre procesamiento.
Esto resultó en una mejora de aproximadamente 0.1 de f1 score.

Los hiperparametros optimizados fueron:

```
{'classifier__alpha': 0.25,  
'classifier__fit_prior': True,  
'vectorizer__max_features': 9000,  
'vectorizer__ngram_range': (1, 3),  
'vectorizer__sublinear_tf': True}
```

Al ver las métricas de Train y Test, no notamos ninguna tendencia a overfitting o a underfitting.

Random Forest

Como segundo modelo entrenamos un modelo Random Forest.

Antes de ingresar los datos al modelo, se realizó una etapa de limpieza crítica para asegurar la calidad de la información. Por un lado, se detectó que el dataset contenía reseñas en idiomas distintos al español, así que decidimos identificar y conservar únicamente las instancias en este idioma.

Como paso fundamental de Ingeniería de Features, y con el objetivo que el modelo procese la información textual de las reseñas, se generaron Embeddings a partir del dataset. Nos parecieron adecuados ya que estos tienen la propiedad de conservar la similitud semántica de los textos, siendo que aquellos con significados similares son representados con vectores matemáticamente cercanos entre sí.

Los parámetros a optimizar fueron:

```
param_dist = {  
    'n_estimators': [100, 120],  
    'max_depth': [15, 20, 25],  
    'max_samples': [0.5, 0.7],  
    'min_samples_split': [5, 10],  
    'max_features': ['sqrt']  
}
```

Es interesante destacar que como paso de post-procesamiento, se optimizó el umbral de decisión por defecto para la métrica F1, realizando un barrido de umbrales en el conjunto de validación, seleccionando un punto de corte personalizado que maximizó la relación entre falsos positivos y falsos negativos.

XGBoost

Para el modelo de XGBoost aplicamos el mismo filtrado por idioma que en el resto de los modelos para conservar únicamente reseñas en español. Luego representamos los textos mediante embeddings semánticos generados con SentenceTransformer("hiiamsid/sentence_similarity_spanish_es"), ya que este modelo captura similitud de significado entre frases en español y mejora el rendimiento de los modelos basados en árboles. Guardamos los embeddings en archivos .npy para evitar recomputarlos en ejecuciones posteriores.

Antes de entrenar, estandarizamos los embeddings con StandardScaler y codificamos la variable objetivo con LabelEncoder. Dividimos los datos en train/validación con estratificación para mantener la proporción de clases.

Entrenamos un XGBClassifier con búsqueda de hiperparámetros mediante GridSearchCV, utilizando validación cruzada estratificada (K=4) y optimizando F1-macro. Los hiperparámetros explorados incluyeron n_estimators, max_depth, learning_rate, subsample, colsample_bytree y gamma:

```
param_grid = {  
    "n_estimators": [50, 100],  
    "max_depth": [5, 8],  
    "learning_rate": [0.5, 0.1],  
    "subsample": [0.8, 1.0],  
    "colsample_bytree": [0.8, 1.0],  
    "gamma": [0, 0.5]  
}
```

La búsqueda permitió seleccionar el mejor modelo, que luego evaluamos con accuracy, precision, recall y F1, tanto en train como en validación. Finalmente, utilizamos el mejor estimador para generar las predicciones del conjunto de test y construir la submission para Kaggle.

Red Neuronal

Para la red neuronal utilizamos el modelo BERT ya entrenado por el Departamento de Ciencias de la Computación de la Universidad de Chile. Además, utilizamos Keras y Tensorflow.

Para este modelo no hicimos ningún pre procesamiento además del filtrado por idioma, debido a que BERT ya con el tokenizer procesa todo. Además, hacerle un pre procesamiento como por ejemplo pasar a minúsculas arruinaría el entrenamiento ya realizado debido a que el modelo es case-sensitive.

En primera instancia tokenizamos los conjuntos de train, validación y test, para luego convertir los primeros 2 en tensores para que BERT los procese.

Además, utilizamos Adam como optimizador y Early Stopping para tratar de reducir el overfitting y mejorar el tiempo entrenamiento.

Para esto último además forzamos al entorno a utilizar la GPU, reduciendo drásticamente el tiempo de entrenamiento.

La búsqueda de hiperparametros fue manual y estos fueron con los que obtuvimos los mejores resultados:

```
BATCH_SIZE = 16  
MAX_LEN = 128  
EPOCHS = 3  
LEARNING_RATE = 2e-5  
NUM_LABELS = 2  
PATIENCE = 2
```

Ensamble de Modelos

Como tercer y último modelo, implementamos una arquitectura de Ensamble del tipo Stacking Classifier.

La arquitectura se compuso de dos niveles:

Nivel Base (Base Learners): Se utilizaron dos modelos de naturaleza distinta para asegurar diversidad en las predicciones:

- Un Random Forest restringido en profundidad para evitar hojas con pocas muestras.
- Un XGBoost configurado con alta penalización (L1/L2) para corregir los errores residuales sin complejizar excesivamente el modelo.

Nivel Meta (Meta Learner): Las predicciones de los modelos base alimentaron a una Regresión Logística, encargada de aprender cuál de los dos modelos base es más confiable para cada tipo de reseña.

Para el desarrollo del modelo de Ensamble, la Ingeniería de Features se construyó sobre la base sólida de los Embeddings utilizados anteriormente, pero incorporando transformaciones adicionales necesarias para la arquitectura de Stacking.

Si bien conservamos los Embeddings como fuente principal de información, identificamos que la combinación de modelos heterogéneos requería un tratamiento especial de los datos numéricos. Por ello, se introdujo una etapa de Estandarización (StandardScaler) previa al entrenamiento.

Finalmente, al igual que en modelos anteriores, se aplicó una optimización del umbral de decisión.

Cuadro de Resultados

Realizar un cuadro de resultados comparando los modelos que entrenaron seleccionando los que a su criterio obtuvieron la mejor performance en cada caso. Deben indicar cuál es el que seleccionaron como mejor predictor de todo el TP. Confeccionar el siguiente cuadro con esta información:

Modelo	F1 Score	Precision	Recall	Accuracy	F1-Kaggle
Bayes Naive	0.855	0.84	0.871	0.853	0.74808
Random Forest	0.841	0.847	0.836	0.843	0.79064
XgBoost	0.862	0.864	0.860	0.863	0.77086
Red Neuronal	0.865	0.789	0.957	0.854	0.82321
Ensamble	0.848	0.855	0.842	0.85	0.79041

Conclusiones generales

El análisis exploratorio fue clave para detectar las características del dataset que nos podrían dar problemas en el futuro, especialmente la presencia de reseñas en otros idiomas, lo que justificó el filtrado previo al entrenamiento. También permitió confirmar que el dataset estaba prácticamente balanceado y que el filtrado no introdujo un desbalance significativo.

Hacer un preprocesamiento distinto según el modelo resultó muy útil. En Naive Bayes, técnicas como la normalización, el ajuste de stopwords y la lematización mejoraron claramente el rendimiento. En Random Forest, XGBoost y el Ensamble, el uso de embeddings en español permitió capturar mejor la información semántica del texto. En contraste, BERT funcionó mejor sin preprocesamiento adicional, ya que su tokenizer está diseñado para trabajar directamente con texto crudo.

Una de las conclusiones que logramos sacar con el cuadro de resultados es que si vemos el f1 score que nos dio en el colab entre XGBoost y la Red Neuronal, podemos ver que son muy similares por lo que podríamos esperar una similitud en su f1 score en Kaggle. Sin embargo, esto no fue así. Podemos ver una gran diferencia entre sus f1 de kaggle y mas aun en la diferencia que tenemos entre el f1 y el f1 de kaggle en el XGBoost. Por lo tanto, podemos decir que el modelo de XGBoost se encuentra overfitteado.

En términos de resultados, el modelo que obtuvo mejor desempeño en test fue la Red Neuronal (BERT) con un F1 de 0.865, y también fue el mejor en Kaggle (0.82321). Sin embargo, el modelo más simple y rápido de entrenar fue Naive Bayes, que aun así logró un rendimiento bastante competitivo, lo que lo hace una buena alternativa en contextos con recursos limitados.

Por lo tanto, BERT es adecuado si se dispone de GPU o infraestructura suficiente, mientras que modelos como XGBoost o Naive Bayes pueden ser preferibles cuando se prioriza la rapidez o los costos computacionales.

Finalmente, algunas mejoras que habíamos planteado para un futuro desarrollo serían explorar transformers especializados en español (como BETO), realizar una búsqueda más amplia de hiperparámetros en XGBoost, optimizar umbrales de decisión en más modelos y aplicar técnicas de data augmentation para mejorar la robustez del sistema.

Tareas Realizadas

Teniendo en cuenta que el trabajo práctico tuvo una duración de 9 (nueve) semanas, le pedimos a cada integrante que indique cuántas horas (en promedio) considera que dedicó semanalmente al TP

Integrante	Principales Tareas Realizadas	Promedio Semanal (hs)
Mariano Merlinsky	Bayes Naive, Random Forest, Embeddings, Redes Neuronales	6
Facundo Calderán	Bayes Naive, XGBoost, Ensemble de Modelos (XGB, RF y Regresion Logistica)	4
Benjamín Castellano	Random Forest, Ensemble de Modelos, distintos preprocesamientos de datos para los modelos en general.	4
Tomás Pons	Bayes Naive, Random Forest y Ensemble de modelos	4